

SIMATS ENGINEERING

ASSESSMENT TOOL - II : DESIGN TASK

Course Code	ECA1422
Course Name	Embedded Systems
Course Outcome	CO1
Assessment Tool	Design Task
Question No.	7, 8
Question Title	7) Develop a formal FSM- Based design for an automated railway level-crossing controller. apply more/many modelling to specify all states, transitions, inputs and outputs. Perform reachability analysis to prove that Barrier never raised while a train is present and demonstrate how formal completeness checking prevents undesired behavior that could lead to safety-critical failures in the real system. 8) Design a formal Petri net model for multi-task embedded system where three tasks share two critical resources a CAN Bus controller and SPI flash memory. Apply Petri Net firing rules to demonstrate the deadlock reachable marking Prove which liveness property is violated. evaluate two formal deadlock Prevention Strategies. Justify which strategy is safer for a hard-real time embedded application.
Maximum Marks	100
Bloom's Taxonomy Level	BTL 5

Evaluation Rubric (100 Marks)

Criteria	Weightage	Level 4	Level 3	Level 2	Level 1	Marks Awarded
Understanding of Design	20	☐ 20	☐ 16	☒ 12	☐ 8	12
Evaluation of Design	25	☐ 25	☐ 20	☒ 15	☐ 10	15
Justification	20	☐ 20	☐ 16	☒ 12	☐ 8	12
Suggestions for Improvement	20	☐ 20	☐ 16	☒ 12	☐ 8	12
Written Communication	15	☐ 15	☐ 12	☐ 9	☒ 6	6
Total	100					57/100

Code of Conduct:

I S. Poovitha (192312183) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S. Poovitha

Develop a formal FSM-Based design for an automated railway level-crossing controller. Apply moore/mealy modelling to specify all states, transitions, inputs, and outputs. Perform reachability analysis to prove that the barrier is never raised while a train is present, and demonstrate how formal completeness checking prevents undefined behaviour that could lead to safety-critical failures in the real system

Aim:

Design a finite state machine (FSM) for an automated railway level-crossing controller using moore/mealy modelling and verify its safety.

Inputs

- i) T = Train detected (1 = yes, 0 = No)
- ii) C = Train has cleared the crossing (1 = yes, 0 = No)

outputs

- i) B = Barrier position (up/down)
- ii) W = Warning signal (ON/OFF)

States

State	Description	Barrier	Warning
S ₀	Idle (No Train)	up	OFF
S ₁	Train detected	down	ON
S ₂	Train crossing	down	ON
S ₃	Train cleared	up	OFF

State Transitions

- $S_0 \rightarrow S_1$: Train detected
- $S_1 \rightarrow S_2$: Train reaches crossing
- $S_2 \rightarrow S_3$: Train clears crossing
- $S_3 \rightarrow S_0$: Return to idle

Moore Model

Outputs depend only on the current state

- S_0 : Barrier up
- S_1 : Barrier down
- S_2 : Barrier down
- S_3 : Barrier up

Mealy Model

Outputs depend on both the present state and inputs, allowing faster response when the train is detected.

Reachability Analysis

Reachability analysis verifies every state can be reached only through valid transitions.

Safety conditions:

- While train present = 1, the FSM can only remain in S_1 or S_2 , where the barrier is always down.
- No transition exists from S_1/S_2 directly to Barrier up.
- Therefore, the barrier can never be raised while a train is present.

Formal Completeness checking
Completeness checking ensures:

Every state has a defined transition for every input.

No undefined or illegal states exist

The controller always moved to a valid state this prevents:

- i) Barrier opening unexpectedly
- ii) Controller changing in an undefined state
- iii) Safety-critical accidents.

Conclusion

The BSM satisfies safety requirements. Reachability analysis guarantees safety operation, while completeness checking eliminates undefined behavior and improves railway crossing reliability.

Design a Formal Petri Net model for a multi-task embedded system where three tasks share two critical resources a CAN Bus controller and an SPI flash memory.

Apply Petri Net firing rules to demonstrate the deadlock reachable marking, prove which liveness property is violated, and evaluate two formal deadlock prevention strategies. Justify which strategy is safer for a hard-real time embedded application.

Aims:

Develop a Petri net model for three embedded tasks sharing two critical resources and analyse deadlock system

Tasks:

- i) T_1
- ii) T_2
- iii) T_3

Resources:

- i) $R_1 = \text{CAN Bus controller}$
- ii) $R_2 = \text{SPI flash memory}$

Petri Net Components

Places

- i) Idle T_1
- ii) Idle T_2
- iii) Idle T_3
- iv) CAN Available
- v) SPI Available
- vi) Task Executing

Transitions

- i) Request CAN
- ii) Request SPI
- iii) Execute Task
- iv) Release CAN
- v) Release SPI

Deadlock Reachable Marking

Example:

- i) T_1 acquires CAN and waits for SPI
- ii) T_2 acquires SPI and waits for CAN
- iii) T_3 waits for Both resources.

No Transitions can fire because each task waits for another resource.

This marking is a deadlock state

Liveness Property Violated

The violated property is Liveness

Reason: Some transitions can never fire again after deadlock, meaning the system cannot continue execution.

Deadlock Prevention Strategies

Strategy 1: Resource ordering

- All tasks request resources in the same order.
- Example:

- i) First CAN
- ii) Then SPI

This removes circular waiting.

Strategy 2: Resource Allocation Control

- i) Allocate both resources together
- ii) If both are unavailable, the task waits without holding any resource.

This prevents partial resource allocation.

Evaluation

Resource ordering is:

- i) Easier to implement
- ii) Lower overhead
- iii) Predictable Timing
- iv) Suitable for hard real-time systems

Resource allocation control:

- i) Safer than unrestricted allocation

is slightly higher execution overhead

Best Strategy:

Resource ordering is preferred because:

- i. Eliminates circular wait.
- ii. Prevents deadlock.
- iii. Provides deterministic Behavior.
- iv. Meets hard real-time embedded system requirements.

Conclusion:

Deadlock is not formally modeled concurrent task execution. Deadlock analysis identifies unsafe markings while resource ordering ensures liveness and reliable operation for safety-critical embedded systems.